

Programming Assignment 2—Natural Loop Detection & CFG

Edge Profiling

Marc Lin

1 Natural Loop Detection

From the definition of natural loop, we first implemented *dfsFindBackEdge* for each Basic Block, and I used iterative-DFS for all the functions that are required to be implemented in DFS manner. The important point is for retreating edge B→A, A is immediate successor of B, even if A is ancestor of B.

```
// iterate through immediate successors (1-level)
// check if there is retreating edge
for (succ_iterator SI = succ_begin(curr), SE = succ_end(curr); SI != SE; ++SI) {
    BasicBlock *succ = *SI;
    // retreating edge points to original target (mHeader)
    // because edge (B,A) means A is also B's successor, even if A is earlier in the CFG
    if (succ == mHeader) {
        if (mDT->dominates(succ, curr)) {
            findNaturalLoop(curr);
        }
    }

    // unvisited successor
    if (visited.find(succ) == visited.end()) {
        st.push(succ);
    }
}
```

After recognition of back edge (A dominates B), we then called *findNaturalLoop* to record natural loop in *backEdgeLoopMap*. We also called helper function *getDominatedBlocks* and *dfsReachable*.

```
void NaturalLoops::findNaturalLoop(BasicBlock *backedge)
{
    // PA2: Implement
    vector<BasicBlock *> dominatedBlocks;
    getDominatedBlocks(mHeader, dominatedBlocks);
    std::pair<string, string> key = std::make_pair(mHeader->getName(), backedge->getName());
    for (auto &bb : dominatedBlocks) {
        // natural loop = nodes that can reach backedge without going through header + header
        if (dfsReachable(bb, backedge, mHeader, set<BasicBlock*>())) {
            // store the backedge into backEdgeLoopMap
            backEdgeLoopMap[key].insert(bb->getName());
        }
    }
    // add header and backedge to the loop
    backEdgeLoopMap[key].insert(mHeader->getName());
    backEdgeLoopMap[key].insert(backedge->getName());
}
```

In *dfsReachable*, we only keep visiting unvisited successor, and the exclude condition should be written in here. If we traverse to the *excludeBlock*, it probably can be bypass, find another route to reach B. so the condition check should be put in unvisited check instead of the stack top check.

```
while (!st.empty()) {
    BasicBlock *curr = st.top();
    st.pop();

    if (curr == target) {
        return true;
    }
    if (visitedBlocks.find(curr) != visitedBlocks.end()) {
        continue;
    }
    visitedBlocks.insert(curr);

    for (succ_iterator SI = succ_begin(curr), SE = succ_end(curr); SI != SE; ++SI) {
        BasicBlock *succ = *SI;

        // unvisited successor (excluding excludeBlock)
        if (succ != excludeBlock && visitedBlocks.find(succ) == visitedBlocks.end()) {
            st.push(succ);
        }
    }
}
```

2 Dynamic Edge Profiling

For the naïve profiling, we iterate every basic block to instrument, so we use a for loop to wrap-up most operations. *logdest* should be implemented first, since *logdest* is the one that truly records edge source from *last_src*, and *logsrc* is to update *last_src* using store instruction.

```

// instrument each basic block with logsrc/logdest calls
IntegerType *i32Ty = IntegerType::getInt32Ty(ctx);
IntegerType *i64Ty = IntegerType::getInt64Ty(ctx);
ConstantInt* numBlocksConst = ConstantInt::get(IntegerType::getInt64Ty(ctx), data.numBlocks);
for (auto &bb : F) {
    unsigned id = data.blockId[&bb];
    IRBuilder<> builder(&bb);
    // logdest step 1: insertion point
    builder.SetInsertPoint(bb.getFirstNonPHI());

    Value *lastSrc = builder.CreateLoad(data.lastSrcGlobal); // member variable
    Value *lastSrc64 = builder.CreateZExt(lastSrc, i64Ty); // zero to extend

    // logdest step 2: index = last src * numBlocks + id
    Value *idConst64 = ConstantInt::get(i64Ty, id);
    Value *mul = builder.CreateMul(lastSrc64, numBlocksConst);
    Value *index = builder.CreateAdd(mul, idConst64);

    // logdest step 3: obtain a pointer to the corresponding counter entry in edge counts by GEP
    Value *gepIdx[] = {ConstantInt::get(i64Ty, 0), index};
    Value *counterPtr = builder.CreateGEP(data.edgeMatrix, gepIdx);

    // logdest step 4: create IR to load the counter value, add 1, and store the incremented value back.
    Value *count = builder.CreateLoad(counterPtr);
    Value *newCount = builder.CreateAdd(count, ConstantInt::get(i64Ty, 1));
    builder.CreateStore(newCount, counterPtr);

    // logsrc step 1:
    builder.SetInsertPoint(bb.getTerminator());
    ConstantInt *idConst32 = ConstantInt::get(i32Ty, id); // not null if type is consistent
    builder.CreateStore(idConst32, data.lastSrcGlobal);
}

```

For the optimized profiling, we first implemented Kruskal Algorithm on weighted edge, which can help us build Max Spanning Tree. The algorithm is edge-based, every time we choose the biggest weighted edge in the tree, until there's no edge that doesn't cause a cycle. This is done by *UnionFind*, and we added the remain edges into *non_maxst_edges*.

After Kruskal scan, we separated all dest and src in *non_maxst_edges* into 2 sets: *needsLogDest*, *needsLogSrc* to discriminate which basic block should be instrumented. Note that blocks end with return statement should be instrumented as well. After this, we do the same iteration with naïve solution, but we add *if (needsLogDest.count(&bb))* and *if (needsLogSrc.count(&bb))* to reduce profiling overhead.

```

// insert selective instrumentation, only in non_maxst_edges
std::set<BasicBlock*> needsLogDest;
for (const auto& e : data.non_maxst_edges) {
    needsLogDest.insert(e.dest); // dest of all non-maxst edges
}
for (BasicBlock &bb : F) {
    if (isa<ReturnInst>(bb.getTerminator())) { // exit block needs logdest
        needsLogDest.insert(&bb);
    }
}
std::set<BasicBlock*> needsLogSrc;
for (BasicBlock &bb : F) {
    for (succ_iterator SI = succ_begin(&bb), SE = succ_end(&bb); SI != SE; ++SI) {
        BasicBlock *succ = *SI;
        if (needsLogDest.count(succ)) { // if any of the successors needs logdest, then this block needs logsrc
            needsLogSrc.insert(&bb);
        }
    }
}

```

3 Efficiency

Tests	Naïve (sec)	Optimized (sec)
01	0.382	0.275
02	0.602	0.332
03	0.674	0.502
04	0.415	0.317
05	0.513	0.266
06	0.748	0.473
07	0.598	0.363
08	0.045	0.032
09	0.360	0.256
10	0.034	0.027
11	0.450	0.367
12	0.415	0.236
13	0.562	0.230
14	0.050	0.041
15	0.043	0.038
16	0.048	0.041
17	0.872	0.679
18	0.457	0.235
19	1.104	0.546
20	0.096	0.079
Average	0.421	0.268

In all testcase, optimized profiling is better than naïve approach. By only instrument on colder path, optimized profiling reduced the operations at hot paths, such as load, store and GEP. Naïve approach instrument all blocks, which cause profiling overhead on those hot paths, while optimized approach already filter them by Kruskal scan, therefore we only instrument on cold path (non-MaxST edges), this makes optimized CFG edge profiling better.